Автономная некоммерческая организация высшего образования
«Университет Иннополис»
(АНО ВО «Университет Иннополис»)

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 – «Информатика и вычислительная техника»

GRADUATION THESIS
(BACHELOR'S GRADUATE THESIS)
Field of Study
09.03.01 – «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»

Тема / Topic	**Обход NAT: экспериментальное исследование на контролируемом испытательном стенде / NAT Traversal: An Experimental Study on a Controlled Testbed**

Работу выполнил / Thesis is executed by	**Майоров Даниил Валерьевич Mayorov Daniil**	подпись / signature
Руководитель выпускной квалификационной работы / Graduation Thesis Supervisor	**Андрей Садовых Andrey Sadovykh**	подпись / signature
Консультанты / Consultants	**Пьер-Филипп Браун Pierre-Philipp Braun**	подпись / signature

Иннополис, Innopolis, 2026

Contents

1 Introduction **3**

1.1 Relevance 3

1.2 Statement of the Problem 4

1.3 Purpose and Objectives of the Work 4

1.4 Research Questions 5

1.5 Field and Methodology of Research 5

1.6 Expected Results 6

1.7 Thesis Structure 6

2 Literature Review **7**

2.1 NAT Basics 7

2.2 NAT Behavior and Classification 8

2.3 Direct Connection Problem Behind NAT 10

2.4 STUN 10

2.5 Hole Punching 11

2.6 TURN 11

2.7 ICE and IPv6 12

2.8 Conclusions on the Chapter 13

Bibliography cited 14

Chapter 1

Introduction

1.1 Relevance

Nowadays, almost all applications work with the Internet: video calls, video conferences, multiplayer games, chats. Some of the applications often tend to transfer data directly between devices, because in this way there is a reduction in latency. But due to the lack of IPv4 addresses, many devices do not have a public IP, as they are placed behind routers that use NAT (Network Address Translation) [1].

NAT allows devices on a private network to use the Internet by rewriting the IP and port in packets. In this way, a device can access the Internet, but devices from the external network cannot initiate a connection to a device from the internal network. The availability of a device is determined by NAT settings such as mapping and filtering rules [1], [2].

But if two devices are behind a NAT, this does not mean that they will not be able to communicate with each other. Several mechanisms are used to establish connections between hosts behind NAT. STUN allows a host to discover the public IP address and port observed by a STUN server. TURN provides a relay when a direct connection cannot be established. ICE uses STUN and TURN to gather

and test connection candidates and select a working communication path [3], [4], [5].

1.2 Statement of the Problem

The main problem is that two devices behind different NAT cannot establish a direct connection without additional tools. The problem arises both in the ignorance of the port and the public IP of the interlocutor, and in the possible setting of NAT filtering, which will not allow packets to pass [2], [6].

RFC 4787 and RFC 5128 describe how NATs behave and which traversal techniques exist. They do not show what ICE actually does for a given pair of NATs: which path it picks when one side has a certain mapping and the other a certain filtering, and what happens when the relay is turned off. This thesis tests these cases on a testbed where the NAT type on each side and the availability of the TURN server are set by hand [2], [6].

1.3 Purpose and Objectives of the Work

The purpose of this thesis is to study STUN and TURN mechanisms for NAT traversal and experimentally investigate how they help establish communication between hosts under different NAT configurations. The thesis also aims to explain these mechanisms in a clear and accessible way so that readers with limited knowledge of NAT traversal can understand them and reproduce the experiments in an isolated environment.

In the course of the work, answers to the following tasks are considered:

1. Learn how NAT works, understand the different types of NAT settings.

2. Learn basic NAT traversal techniques.

3. Learn how STUN works.
4. Learn how TURN works.
5. Learn what ICE is, how it works, and how it chooses the best candidate to connect with.
6. Assemble a test environment of two networks behind NAT and a relay server.

1.4 Research Questions

Question 1: How do STUN and TURN contribute to establishing communication between two hosts behind NAT, and how does their effectiveness depend on NAT mapping and filtering behavior?

Question 2: How does NAT traversal work in practice? If I set up two separate networks with NAT, with a STUN/TURN server situated between them, what do I observe when two hosts attempt to establish a connection, and what changes when I alter the NAT configuration or disable the relay?

Bonus question (optional): What could an attacker do in this configuration: from the outside against the STUN/TURN service, from or via a malicious or misconfigured relay, as an attacker en route, or by exploiting the bypass mechanism itself? Which of these actions can I replicate in my test environment, and which defences are effective?

1.5 Field and Methodology of Research

The work focuses on STUN and TURN mechanisms and includes practical experiments in a virtual environment to investigate their behavior under different NAT configurations, as well as additional experiments will be carried out in

practice in a virtual environment. In a virtual environment, two hosts behind the NAT will be deployed, as well as a server with public addresses that act as STUN and TURN. Ready-made protocol implementations will be used.

1.6 Expected Results

It is planned that this thesis will clearly describe the mechanisms of NAT traversing, starting with an explanation of what NAT is, ending with how ICE selects candidates. In addition to the theoretical part, a reproducible experimental rig will be proposed, as well as conclusions about the behavior of STUN and TURN under different NAT configurations and the effectiveness of direct and relayed communication. This thesis does not propose a new protocol, it consists in the study of existing solutions.

1.7 Thesis Structure

Chapter 2 reviews the RFCs and other work on NAT traversal.

Chapter 2

Literature Review

This chapter first describes how NAT works and why it prevents two nodes from connecting directly. It also describes the basic tools for traversing NAT, namely STUN, hole punching, TURN, and ICE.

2.1 NAT Basics

NAT (Network Address Translation) is a router mechanism for rewriting one IP address and port with another between private and public networks. It is needed because IPv4 addresses are scarce, and many devices access the Internet through the same public address. The problem with NAT is that it drops incoming packets that have not been sent to an outgoing request, so it is impossible to establish a connection from the outside without auxiliary tools [1], [7].

The principle of NAT is to maintain an additional table of the "address:port - address:port" type. When a device sends a packet to the Internet, NAT creates an entry in the table and replaces the sender's address. The same happens with the response that comes to the external address and port: NAT replaces its address with an internal one. The variant in which NAT translates not only addresses, but also ports is called Network Address Port Translation (NAPT). Thanks to NATP, it is possible for several users to share one external address [1].

There are two subtypes of NAT [7]:

1. **Static Address Assignment** - one internal address always corresponds to one external address and does not change over time.

2. **Dynamic Address Assignment** - addresses are assigned dynamically from the pool as needed. In case the address is no longer needed, it is released and can be used by another device.

If a packet arrives at an external address and port that is not in the NAT table, such a packet is discarded, which makes it impossible to initialize the connection by an external node [2].

2.2 NAT Behavior and Classification

NAT has two independent properties: Mapping and Filtering [2].

Mapping determines the mapping of an external port and address to an internal address and port. RFC 4787 defines 3 types [2]:

1. **Endpoint-Independent Mapping (EIM)**: For a given internal address and port, the NAT reuses the same external address and port regardless of the destination.

2. **Address Dependent Mapping (ADM)**: A unique external port is used for each external ip.

3. **Address And Port Dependent Mapping (APDM)**: A new port is allocated for each new IP and port pair.

Filtering is the rules that determine which packets can be sent to internal addresses and which cannot.

There are 3 types [2]:

1. **Endpoint-Independent Filtering (EIF)**: Allows packets from any external address and port if mapping already exists.

2. **Address-Dependent Filtering (ADF)**: Allows packets only from external IP addresses to which the internal host has previously sent packets. The external port may differ.

3. **Address and Port-Dependent Filtering (APDF)**: Allows packets to the internal IP address only if the internal address initiated the connection and there is an exact match between the external IP and port.

For example, a device on a home network with an internal port of 54000 accesses a server on the Internet. Then NAT allocates an external port for it at a public address, for example, 34000, and writes it to a table. A packet arriving at a public address and port 34000 will be forwarded to an internal address and port 54000. This is called mapping. And from whom such packets will be received is decided by the filtering rules.

Previously, NAT was described by the classification RFC 3489, where there were 4 types of NAT: full cone, restricted cone, port restricted cone, and symmetric. But later it was replaced by another standard, RFC 4787, which replaced the classification with two parameters: filtering and mapping. RFC 3489 itself was replaced by RFC 5389, and then by RFC 8489 [2], [3], [8], [9].

1. Full cone = EIM (Mapping) + EIF (Filtering)

2. Restricted cone = EIM (Mapping) + ADF (Filtering)

3. Port Restricted Cone = EIM (Mapping) + APDF (Filtering)

4. Symmetric = APDM (Mapping) + APDF (Filtering)

One of the special cases is CGNAT (Carrier-Grade NAT). CGNAT is the NAT used by Internet providers. In this case, the provider's subscribers share

the provider's public addresses. Subscriber cannot control and configure the provider's NAT [10].

2.3 Direct Connection Problem Behind NAT

To reach a device behind NAT, it is not enough to know its public IP address: you also need an external port. The port is allocated dynamically, and incoming packets to it go through NAT filtering. If both participants are behind NAT, neither knows the external address and port of the other, and each NAT discards unsolicited incoming packets [6].

Usually, the problem is solved in several steps. First, each node learns its external address through STUN. Then the nodes exchange addresses through the signaling server and try to "punch" NAT using the hole punching method. If it fails, the data goes through the TURN relay. The order of attempts and the choice of the final path is organized by ICE. Each mechanism is discussed in more detail in the following sections [5], [6].

2.4 STUN

STUN (Session Traversal Utilities for NAT) serves as a tool for other NAT traversal mechanisms, rather than a standalone solution. The client sends a STUN Binding request to a server on the Internet, to which the server responds with a message with the address and port from which the request came. The address and port correspond to the nearest NAT to the STUN server [3].

STUN only reports the address and does not transmit application data. Knowing your external address does not guarantee that another node will reach it: NAT can still filter incoming packets or allocate a new port for each recipient. Therefore, STUN is used in conjunction with hole punching, TURN, and ICE. It also

allows you to determine the behavior of a specific NAT; this method is described in the experimental RFC 5780 [2], [11].

2.5 Hole Punching

Hole punching allows two nodes behind NAT to establish a direct connection: they send packets to each other at the same time [6].

Suppose clients A and B are behind different NATs, and server S has a public address. The procedure is as follows:

1. Client A asks the server for its external address and port.
2. Client B does the same.
3. Clients exchange the received addresses through the signal server.
4. A and B simultaneously send packets to each other's external addresses. The outgoing packet of each client creates an entry in its NAT, and the counterpacket passes through this record. If all goes well, the connection is established.

The success of the method depends on the mapping and filtering of both NATs. If NAT uses APDM (symmetric NAT in the old terminology), it allocates a new external port to connect to the other client, rather than the one that was reported to the server. Packets from the second client go to the wrong port, and the method fails. With EIM, including in combination with the most restrictive APDF (port restricted cone) filtering, hole punching usually works [2].

2.6 TURN

TURN (Traversal Using Relays around NAT) extends STUN and uses the same message format. The TURN server works as a relay: the client requests it

to allocate a resource, i.e. a public address and port on the server. Through this address, the server sends data between the client and other participants. To prevent the server from forwarding traffic from outsiders, the client sets permissions for partner addresses; overhead can be reduced by using channels. TURN over TCP for partner connections is described in a separate RFC 6062 [4], [12].

TURN is needed when a direct connection is impossible, for example, when one of the participants is behind NAT with APDM. This comes at the cost of additional latency and load on the server through which all session traffic goes [4]. Therefore, TURN is considered a fallback option, not the main path.

2.7 ICE and IPv6

ICE (Interactive Connectivity Establishment) uses STUN and TURN and selects the best available connection methods. First, each participant collects candidates, i.e. addresses at which they can try to connect to it [5].

There are three types of candidates [5]:

- host: the address of the network interface of the device itself;
- server reflexive: the external address and port received from the STUN server;
- relayed: the address allocated on the TURN server.

Participants exchange candidates through the signaling channel. ICE then pairs its candidate and the partner's candidate, orders them by priority, and checks each pair with STUN requests. The best working pair is selected for data transmission. As a rule, the direct path is preferred, and the path through relay is used as a fallback [5].

The RFC 8445 specification is in effect, it replaced RFC 5245. The basic mechanism is complemented by extensions. Trickle ICE allows checks to begin

without waiting for all candidates to be collected. Consent freshness checks periodically confirm that the partner is still ready to accept data. RFC 8421 describes how ICE works on IPv4/IPv6 dual-stack multi-interface nodes [5], [13], [14], [15], [16].

IPv6 has enough addresses, so NAT is usually not needed: the device can have its own global address. However, a direct connection can still be blocked by firewall settings, which by default discard unsolicited incoming traffic. Therefore, hole punching and ICE are also needed in IPv6: ICE iterates through the available options and selects the appropriate one for a particular network, and in dual-stack networks, also between IPv4 and IPv6 candidates [16].

2.8 Conclusions on the Chapter

A review of the literature has shown that the behavior of NAT is conveniently described by two independent properties, mapping and filtering, and that it is on them that the possibility of a direct connection depends. STUN, hole punching, and TURN complement each other: STUN provides an external address, hole punching uses it for a direct connection, and TURN provides communication when a direct path is not possible. ICE combines these mechanisms and chooses the best working path.

Bibliography cited

[1] P. Srisuresh and K. Egevang, "Traditional IP network address translator (Traditional NAT)," IETF, RFC 3022, Jan. 2001. DOI: 10.17487/RFC3022

[2] F. Audet and C. Jennings, "Network address translation (NAT) behavioral requirements for unicast UDP," IETF, RFC 4787, Jan. 2007. DOI: 10.17487/RFC4787

[3] M. Petit-Huguenin, G. Salgueiro, J. Rosenberg, D. Wing, R. Mahy, and P. Matthews, "Session traversal utilities for NAT (STUN)," IETF, RFC 8489, Feb. 2020. DOI: 10.17487/RFC8489

[4] T. Reddy, A. Johnston, P. Matthews, and J. Rosenberg, "Traversal using relays around NAT (TURN): Relay extensions to session traversal utilities for NAT (STUN)," IETF, RFC 8656, Feb. 2020. DOI: 10.17487/RFC8656

[5] A. Keranen, C. Holmberg, and J. Rosenberg, "Interactive connectivity establishment (ICE): A protocol for network address translator (NAT) traversal," IETF, RFC 8445, Jul. 2018. DOI: 10.17487/RFC8445

[6] P. Srisuresh, B. Ford, and D. Kegel, "State of peer-to-peer (P2P) communication across network address translators (NATs)," IETF, RFC 5128, Mar. 2008. DOI: 10.17487/RFC5128

[7] P. Srisuresh and M. Holdrege, "IP network address translator (NAT) terminology and considerations," IETF, RFC 2663, Aug. 1999. DOI: 10.17487/RFC2663

[8] J. Rosenberg, J. Weinberger, C. Huitema, and R. Mahy, "STUN – simple traversal of user datagram protocol (UDP) through network address translators (NATs)," IETF, RFC 3489, Mar. 2003. DOI: 10.17487/RFC3489

[9] J. Rosenberg, R. Mahy, P. Matthews, and D. Wing, "Session traversal utilities for NAT (STUN)," IETF, RFC 5389, Oct. 2008. DOI: 10.17487/RFC5389

[10] S. Perreault, I. Yamagata, S. Miyakawa, A. Nakagawa, and H. Ashida, "Common requirements for carrier-grade NATs (CGNs)," IETF, RFC 6888, Apr. 2013. DOI: 10.17487/RFC6888

[11] D. MacDonald and B. Lowekamp, "NAT behavior discovery using session traversal utilities for NAT (STUN)," IETF, RFC 5780, May 2010. DOI: 10.17487/RFC5780

[12] S. Perreault and J. Rosenberg, "Traversal using relays around NAT (TURN) extensions for TCP allocations," IETF, RFC 6062, Nov. 2010. DOI: 10.17487/RFC6062

[13] J. Rosenberg, "Interactive connectivity establishment (ICE): A protocol for network address translator (NAT) traversal for offer/answer protocols," IETF, RFC 5245, Apr. 2010. DOI: 10.17487/RFC5245

[14] E. Ivov, J. Uberti, and P. Saint-Andre, "Trickle ICE: Incremental provisioning of candidates for the interactive connectivity establishment (ICE) protocol," IETF, RFC 8838, Jan. 2021. DOI: 10.17487/RFC8838

[15] M. Perumal, D. Wing, R. Ravindranath, T. Reddy, and M. Thomson, "Session traversal utilities for NAT (STUN) usage for consent freshness," IETF, RFC 7675, Oct. 2015. DOI: 10.17487/RFC7675

[16] P. Martinsen, T. Reddy, and P. Patil, "Guidelines for multihomed and IPv4/IPv6 dual-stack interactive connectivity establishment (ICE)," IETF, RFC 8421, Jul. 2018. DOI: 10.17487/RFC8421